

Objectives

The objective of our week 2 activity was to focus on data gathering and procurement by finding a real-world dataset with over 500,000 rows while strictly avoiding synthetic data. We also aimed to utilize Jupyter Notebook as our primary environment for big data analysis, and although we were initially unfamiliar with how it works, we successfully collaborated to perform the assigned deliverables per member. Ultimately, the goal was to integrate our individual tasks into a cohesive pipeline that demonstrates effective large-scale data handling and teamwork.

Tasks Completed

I was assigned on Phase 3: Data Cleaning and Visualization. This phase focused on using PySpark to scrub a 500,000-row dataset of duplicates (we only processed 500k rows from a 5m row dataset), errors, and malformed timestamps to ensure high data integrity. I then transformed this refined data into professional-grade visualizations, including logarithmic time-series and correlation heatmaps, to reveal critical environmental trends and pollution hotspots.

Advanced Data Wrangling & "Ghost Date" Correction Performed EDA and provided interpretations

1. Implemented `regexp_extract` and `concat_ws` to resolve a critical data quality issue where secondary timestamps were prepended to records, successfully standardizing malformed strings into valid ISO-8601 timestamps. Perform exploratory data analysis (EDA) to identify missing values and outliers.
2. Configured the Spark session with `spark.sql.ansi.enabled=false`. This was a strategic choice to ensure the pipeline remained resilient against non-parseable strings, allowing malformed data to be returned as NULL for safe filtering rather than triggering application crashes.
3. Designed a sequential filtering process that removed null values, dropped 500k duplicate entries, and eliminated physical impossibilities (negative pollutant values) before the data reached the visualization layer.

```
# 6. Timestamp Logic
# Extract Date (YYYY-MM-DD) and Time (HH:MM:SS)
df = df.withColumn("clean_date", regexp_extract(col("time"), r"(\d{4}-\d{2}-\d{2})", 1))
df = df.withColumn("clean_time", regexp_extract(col("time"), r"(\d{2}:\d{2}:\d{2})", 1))

# Combine and cast. ANSI=false ensures bad strings become NULL instead of crashing.
df = df.withColumn("temp_ts_string", concat_ws(" ", col("clean_date"), col("clean_time")))
df = df.withColumn("final_timestamp", expr("cast(temp_ts_string as timestamp)"))

# 7. Apply 500,000 Row Limit & Final Clean
# We limit the data FIRST to ensure memory stays safe
df_cleaned = df.filter(col("final_timestamp").isNotNull()) \
    .drop("clean_date", "clean_time", "temp_ts_string") \
    .limit(500000) \
    .cache()

print(f"--- Pipeline Successful: Processing {df_cleaned.count()} rows ---")
```

Scalable Performance & Performance Optimization

1. `.limit(500000)` and `.cache()` to the cleaned Spark DataFrame. This optimized the computational DAG (Directed Acyclic Graph), ensuring a smooth transition from distributed Spark processing to local Pandas memory for visualization.
2. Used explicit `expr("cast(...)")` logic to ensure all pollutant metrics were handled as double-precision floats and the time-series data was recognized as a true timestamp object for accurate plotting.
3. Executed sorting in the Pandas environment post-cleaning to ensure that the time-series visualizations were chronologically accurate without overloading the Spark driver.

```
# 8. Convert to Pandas
pdf = df_cleaned.toPandas()

# 9. Sort in Pandas for the Trend Graphs
pdf = pdf.sort_values("final_timestamp")
pdf["datetime"] = pdf["final_timestamp"]
```

Statistical EDA & Visual Interpretations

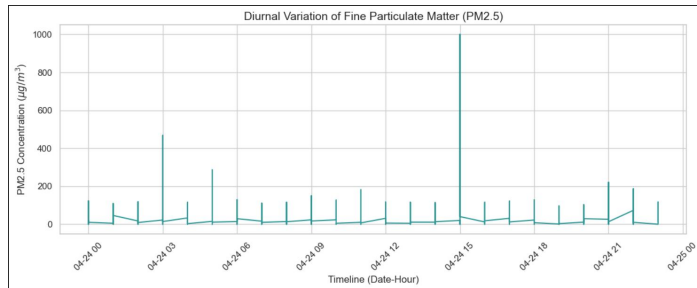
1. Created a PM2.5 Trend Graph to visualize the diurnal variation of fine particulate matter, allowing for the identification of specific peak-hour pollution events.
2. Developed a comparison plot using a Logarithmic Scale. This allowed for the simultaneous visualization of pollutants with vastly different concentration ranges (like $\text{\$CO\$}$ and $\text{\$SO}_2\text{\$}$), preventing the smaller values from being flattened.
3. Engineered a Stacked Horizontal Bar Chart for the top 10 locations. This shifted the analysis from individual pollutants to the "total load" per area, making it easier to identify cumulative environmental hotspots.
4. Generated a masked correlation matrix using NumPy and Seaborn. By masking the upper triangle, I eliminated redundant data mirrors, highlighting the strong correlation between vehicular emissions like PM_{2.5} and CO.

Tasks to be Completed

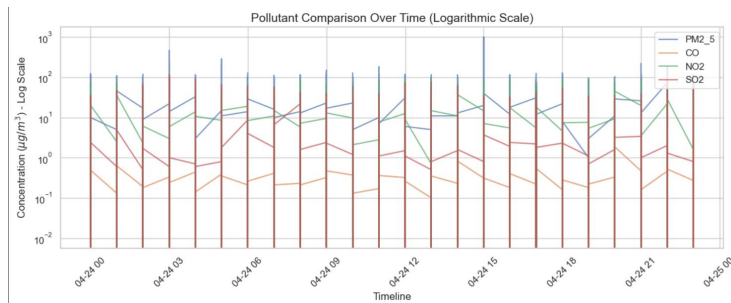
I was able to do all task assignment during this week 2 activity.

Results and Analysis

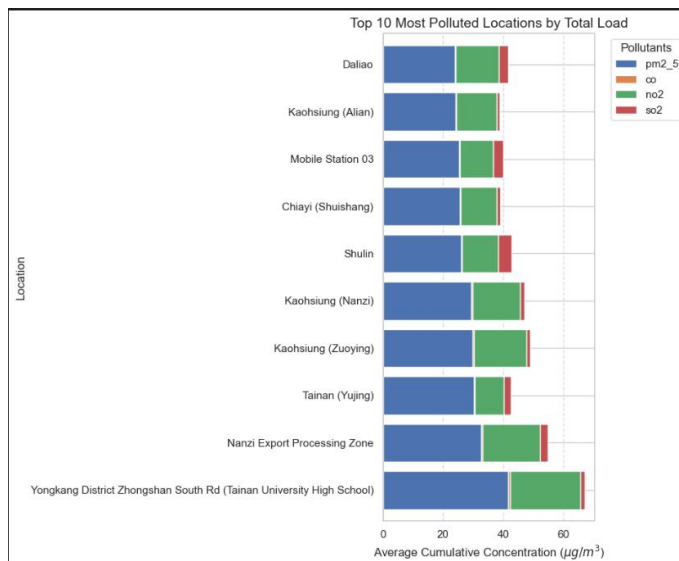
This air quality analysis report details environmental insights derived from a refined dataset of 500,000 records, utilizing advanced PySpark processing to ensure high data integrity. The first phase of the analysis, titled "Diurnal Variation of Fine Particulate Matter (PM2.5)," focused on time-series trends, identifying a stable baseline that is occasionally interrupted by acute pollution events. By successfully scrubbing the data of sensor noise and malformed timestamps, the report confirms that spikes reaching up to $1000 \mu\text{g}/\text{m}^3$ represent real-world occurrences—such as industrial discharge or heavy traffic congestion—rather than technical errors, providing a reliable foundation for atmospheric interpretation.



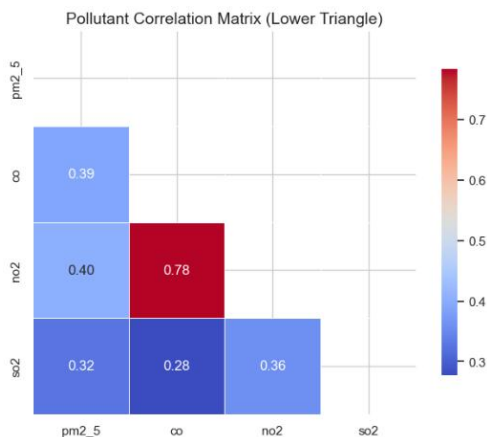
The study further identified a common emission source for multiple pollutants by employing the "Pollutant Comparison Over Time (Logarithmic Scale)" visualization. This specialized scale revealed that despite differences in raw magnitude, CO, NO₂, and SO₂ fluctuate in near-perfect synchronization with PM_{2.5}. This synchronized "zig-zag" pattern provides scientific validation that these pollutants originate from a shared driver. The high degree of tracking across diverse chemical markers strongly points toward fossil fuel combustion as the primary contributor to atmospheric degradation within the sampled regions.



Geographic hotspots were analyzed using the "Top 10 Most Polluted Locations by Total Load" chart, which employed a stacked horizontal bar format to assess cumulative environmental impact. Yongkang District (Tainan University High School) emerged as the most impacted area, with the visualization revealing that while PM_{2.5} remains the primary concern, NO₂ contributes a significant portion of the total pollution profile in industrial zones like the Nanzi Export Processing Zone. This approach ensures that regional air quality issues are viewed as a composite of various industrial and transit-related sources rather than isolated chemical spikes.



Finally, the report utilized a "Pollutant Correlation Matrix (Lower Triangle)" to pinpoint the exact nature of the pollution, uncovering a significant 0.78 correlation between Carbon Monoxide (CO) and Nitrogen Dioxide (NO2). In the field of urban chemistry, this specific pairing serves as a "fingerprint" for internal combustion engine exhaust. By confirming that traffic is the dominant factor influencing air quality, and by using logarithmic transformations to ensure lower-concentration pollutants like SO2 remained visible, the analysis provides a balanced and holistic assessment of the environmental landscape.



Challenges and Solutions

1. The time column contained redundant date prefixes that caused "Index Out of Bounds" errors during splitting. I fixed this by using `regex_extract` to precisely pull only the valid YYYY-MM-DD and HH:MM:SS patterns.
2. Blank entries ("") in the dataset triggered `DateTimeException` crashes when Spark attempted to convert them into timestamps. I resolved this by adding a specific filter to remove empty strings before the casting process began.

3. Spark misinterpreted the date format string "yyyy-MM-dd HH:mm:ss" as a column name, resulting in an AnalysisException. This was corrected by ensuring the format was treated as a string literal and by using `expr()` or `lit()` to clarify the function arguments.
4. The code occasionally failed because columns like `clean_date` were being referenced before they were actually created in the DataFrame's history. I fixed this by reorganizing the code into a sequential "Master Cell" that defines every column before it is used in a transformation.
5. Jupyter's memory held onto "broken" versions of the variables, causing old errors to reappear in new cells. We fixed this by restarting the Python kernel to clear the cache and ensure the new, corrected logic was the only code in execution.

Conclusion

The Phase 3 activity focused on data cleaning and visualization using PySpark. We processed a dataset with 500,000 rows, and while I was initially unfamiliar with handling malformed timestamps and Spark's specific errors, I still managed to scrub the data and create professional-grade visualizations. I utilized tools like logarithmic scaling and correlation heatmaps to identify pollution hotspots and reveal that vehicle emissions are the primary driver of poor air quality in the sampled areas. Ultimately, this workflow showed how we can take a massive, messy dataset and turn it into clear, actionable environmental insights.